CSDX 536

IMAGE & VIDEO ANALYTICS

ASSIGNMENT

Submitted by

AZMAAN PASHA (_____)

Department: B.Tech – AI & DS "A"

CHROMATIC VISION – INTELLIGENT COLOR DETECTION AND OBJECT TRACKING USING IMAGE PROCESSING TECHNIQUES

1. Abstract

Chromatic Vision is an image and video analytics application designed to detect objects of a chosen colour in images and to follow those objects across the frames of a video. The system uses classical image processing techniques rather than deep learning, so it does not require any training data, labelled dataset or GPU.

Every input image or video frame is converted from the BGR colour space to the HSV (Hue, Saturation, Value) colour space. A binary mask of the target colour is produced by HSV thresholding, using two hue ranges for red because red lies at both ends of the hue circle. The mask is cleaned with morphological opening and closing, and contour analysis is used to locate each object, draw its bounding box and calculate its centroid. For videos, a centroid-based tracker links the detections of consecutive frames so that every object keeps a consistent ID.

A Streamlit application with five pages (Overview, Image Analysis, Video Tracking, Analytics & Insights and About Project) has been developed with OpenCV, NumPy, Plotly, Matplotlib and Pillow. Users can upload their own media or use the bundled sample images and videos, select one of six colour presets or a custom range, adjust the processing parameters and download the processed image, binary mask, segmented image, tracked video and CSV reports.

On the bundled sample set the detected object counts matched the known ground truth for all 15 image–colour combinations tested. In the 200-frame sample video the two red balls were tracked with two stable IDs, including through a short occlusion, at an average of 9.35 ms per frame. All 34 automated unit tests and 22 end-to-end interface checks passed.

The project demonstrates the practical use of colour spaces, segmentation, morphology, contour analysis and object tracking. It is developed as an educational prototype; its colour presets are tuned for clearly coloured objects and are not intended for safety-critical use.

2. Introduction

Colour is one of the simplest and most informative visual features of an object. Many practical tasks such as sorting coloured products, following a coloured marker, counting coloured items or locating a ball in a sports clip can be solved by finding the pixels that have a particular colour and grouping them into objects.

Digital images are usually stored in the RGB (or, in OpenCV, BGR) colour space, where the colour and the brightness of a pixel are mixed across three channels. A small change in lighting therefore changes all three values. The HSV colour space separates the colour itself (hue) from its purity (saturation) and its brightness (value), which makes it much easier to describe a colour with a fixed range of numbers.

This project focuses on colour-based object detection and tracking using image processing techniques. The system analyses uploaded images and video frames to identify objects of a selected colour, removes noise from the result with morphological operations, measures each object with contour analysis and follows moving objects with a centroid tracker.

OpenCV performs the image and video processing, NumPy handles the pixel arrays, and Streamlit provides an interactive interface for uploading media, choosing parameters and viewing results. Plotly and Matplotlib are used to present real analytics computed from each run.

The project demonstrates how image and video analytics can be applied to colour detection and object tracking without the cost of training a deep learning model.

3. Problem Statement

Identifying and counting objects of a particular colour by eye is slow and error-prone, especially when there are many objects in an image or when the objects move through a long video.

Simple thresholding in the RGB colour space is not reliable because brightness and colour are mixed, so the same object can produce very different RGB values under different lighting. Red objects are a particular difficulty, because their hue values fall on both sides of the 0°/360° boundary of the hue circle.

Therefore, a computer vision-based system is developed to detect objects of a selected colour in uploaded images and videos, using the HSV colour space and a dual hue range for red.

The system aims to segment the target colour, remove noise, locate each object with a bounding box and centroid, assign consistent IDs to moving objects in videos, and report measured statistics such as object count, colour coverage, detection frequency and processing time.

4. Objectives

The main objectives of the project are:

- To develop a colour detection system based on the HSV colour space.
- To provide predefined HSV presets for red, orange, yellow, green, blue and purple, and a custom range option.
- To detect red correctly using two hue ranges that are combined with a logical OR.
- To remove noise from the binary mask using erosion, dilation, opening and closing.
- To locate objects with contour detection and to draw bounding boxes, labels and centroids.
- To process uploaded videos frame by frame and track moving objects with consistent IDs.
- To calculate real analytics: object count, coverage, colour distribution, detections per frame and processing time.
- To allow the processed image, mask, segmented image, tracked video and CSV reports to be downloaded.
- To develop an interactive, easy-to-use application using Streamlit.
- To verify the system with automated tests against a sample set with known ground truth.

5. Existing and Proposed Approach

5.1 Existing approach: Colour-based inspection is often done manually, by looking at images or video and counting objects by eye. Simple automated approaches threshold the RGB values directly, while modern approaches train deep learning detectors. These approaches have certain limitations:

- Manual counting and following of objects requires continuous human attention.
- Analysing many images or long videos manually is time-consuming and inconsistent.
- RGB thresholds are sensitive to lighting, because colour and brightness are mixed in every channel.
- A single hue interval cannot capture red, which wraps around the end of the hue circle.
- Deep learning detectors need a labelled dataset, training time and often a GPU, which is excessive for detecting a known colour.

5.2 Proposed approach: The proposed system uses the HSV colour space and classical image processing. Each image or frame is converted to HSV and thresholded with cv2.inRange using the selected colour preset. For red, two masks (hue 0–10 and 170–179) are combined. Morphological opening removes small noise and closing fills small holes, after which cv2.findContours locates each object. Objects smaller than

a minimum area are ignored, and the remaining objects are measured with a bounding rectangle and image moments.

For videos, OpenCV reads the file frame by frame and the same detection pipeline is applied to every frame. A centroid tracker matches each detection to the nearest existing track, so each object keeps the same ID while it moves. The annotated frames are written to WebM and MP4 output videos. The Streamlit application displays the results, charts and statistics and provides download buttons. No training is required, and the whole system runs on an ordinary CPU.

6. Technologies and Tools Used

Technology	Purpose	Role in the project
Python 3.12	Programming language	Implements the project logic.
OpenCV	Computer vision library	Colour conversion, thresholding, morphology, contours, video reading and writing.
NumPy	Numerical computing library	Stores images as arrays; computes distances in the tracker and pixel statistics.
Streamlit	Web application framework	Provides the five-page interface for uploading media and viewing results.
Plotly	Interactive charting library	Draws the analytics charts (distribution, objects per frame, trajectories).
Matplotlib	Plotting library	Draws the hue spectrum chart and the report figures.
Pillow	Image processing library	Reads image format information for uploaded files.
pytest	Testing framework	Runs the 34 automated unit tests.
Playwright	Browser automation tool	Runs the end-to-end interface checks and captures the screenshots.
Git and GitHub	Version control	Stores and publishes the project source code.

7. System Requirements

7.1 Hardware requirements:

- Computer or laptop capable of running Python 3.10 or later.
- No GPU is required; all processing runs on the CPU.
- Minimum 4 GB RAM; more is useful for long or high-resolution videos.
- Sufficient storage space for the uploaded and processed video files.
- The project was developed and tested on a Windows 10 PC.

7.2 Software requirements:

- Python 3.12 virtual environment (Python 3.10 or later is supported).
- Streamlit 1.50 or later.
- OpenCV (opencv-python-headless) 4.9 or later.
- NumPy 1.26 or later.
- Plotly 5.20 or later and Matplotlib 3.8 or later.
- Pillow 10.0 or later.

- pytest (for running the tests).
- A modern web browser (Chrome, Edge or Firefox).

8. System Architecture and Workflow

The proposed system follows a sequence of operations from media input and colour selection to detection, tracking and result presentation.

8.1 System architecture

The system is organised in three layers and consists of the following major components:

1. **Presentation layer:** app.py sets up the page and navigation; the components and views packages build the sidebar, colour picker and the five pages; styles/main.css provides the dark theme.
2. **Media input:** Users upload an image (JPG, JPEG, PNG) or a video (MP4, AVI, MOV), or select a bundled sample.
3. **Colour selection:** color_ranges.py provides the HSV presets and builds the dual range for red or a custom range from the sliders.
4. **Image processing:** image_processing.py converts to HSV, creates the mask, applies morphology and finds the objects.
5. **Video processing:** video_processing.py reads each frame, runs the detection pipeline, draws the overlay and writes the output videos.
6. **Object tracking:** object_tracking.py assigns and maintains object IDs with a centroid tracker.
7. **Analytics:** analytics.py computes colour distribution and hue histograms and builds the charts.
8. **Foundation:** OpenCV, NumPy, Plotly and Matplotlib do the computation and visualisation; st.session_state stores the results and statistics of the current session.

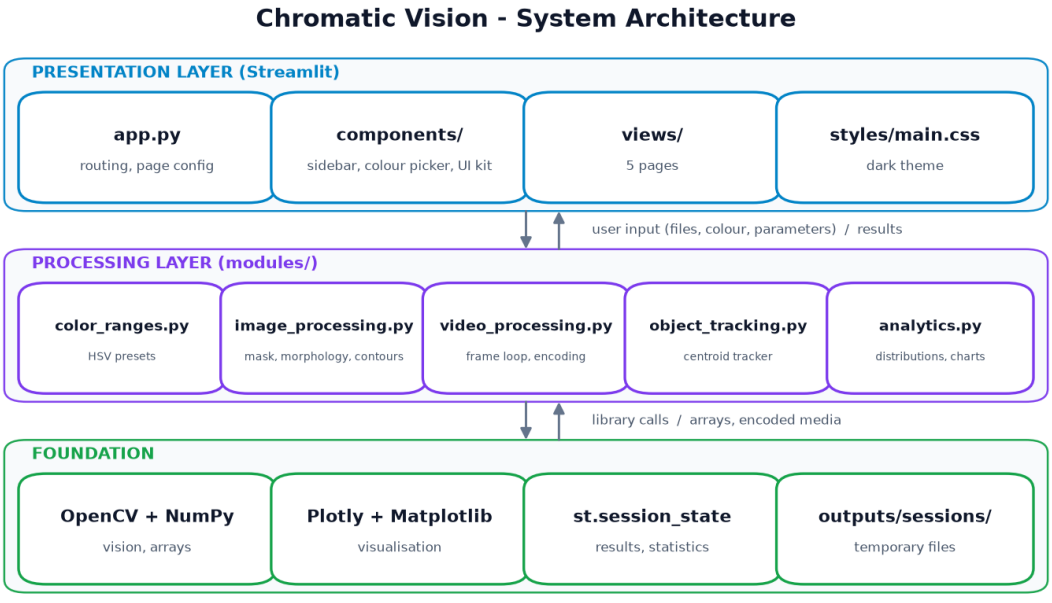


Figure 1: System Architecture of Chromatic Vision

8.2 Workflow

The system follows the processing sequence below:

1. Open the application and choose Image Analysis or Video Tracking.
2. Upload an image or video, or select a bundled sample.
3. Validate the file; unreadable or empty files show an error message.

4. Select the target colour preset or define a custom HSV range.
5. Adjust the processing parameters (kernel size, minimum area and, for video, tracking settings).
6. Convert the image or frame from BGR to HSV with cv2.cvtColor.
7. Create the binary mask with cv2.inRange (two ranges combined for red).
8. Clean the mask with morphological opening followed by closing.
9. Find the external contours and discard those below the minimum area.
10. Calculate the bounding box, area and centroid of each object.
11. For video, match the centroids to existing tracks and assign IDs.
12. Draw the bounding boxes, labels, centroids and trails on the output.
13. Repeat steps 6–12 for every frame of a video and write the output video.
14. Compute the statistics and charts and display them with the download options.

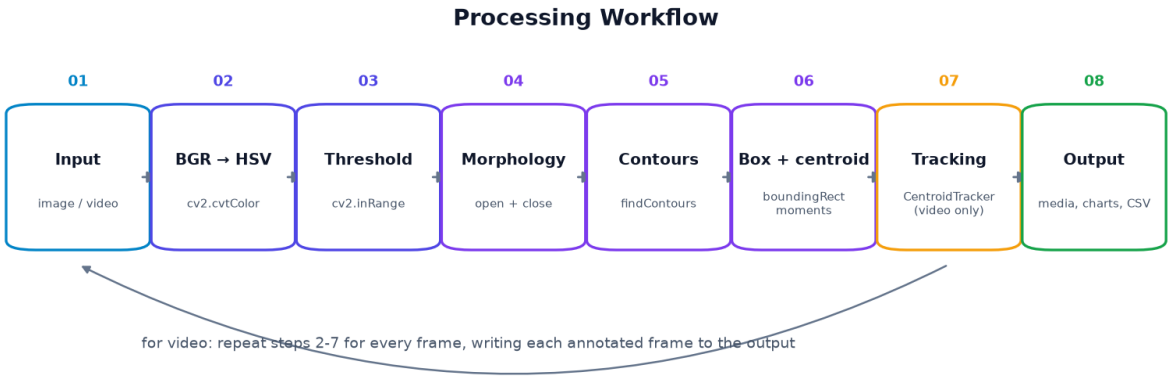


Figure 2: Processing Workflow

9. Methodology and Algorithm

9.1 Dataset details

Colour detection with HSV thresholding is a rule-based method, so no training dataset is required. To test the system objectively, a sample dataset with known ground truth was generated with the script scripts/generate_samples.py. Each sample contains solid, clearly coloured shapes with a known number of objects of each colour, and the expected counts are stored in assets/sample_ground_truth.json. The same files are used by the demo mode of the application and by the automated tests.

File	Type	Resolution	Frames / FPS	Content
shapes_scene.png	Image (PNG)	1280 × 800	–	Circles, squares and triangles in six colours on a dark background.
ball_collection.jpg	Image (JPG)	1000 × 640	–	Red, blue and green balls with shading (JPEG compressed).
neutral_scene.png	Image (PNG)	900 × 600	–	Only white, grey and black shapes; used as a negative test.
moving_objects.mp4	Video (MP4)	640 × 360	200 / 25 fps	Two red balls, one blue, one green and one yellow object moving; includes a short occlusion.
orbiting_ball.avi	Video (AVI)	480 × 360	120 / 30 fps	One orange ball moving on a circular path.

The ground truth number of objects of each colour is as follows:

Sample	Red	Orange	Yellow	Green	Blue	Purple
shapes_scene.png	3	2	2	3	3	2
ball_collection.jpg	4	0	0	2	3	0
neutral_scene.png	0	0	0	0	0	0
moving_objects.mp4	2	0	1	1	1	0
orbiting_ball.avi	0	1	0	0	0	0

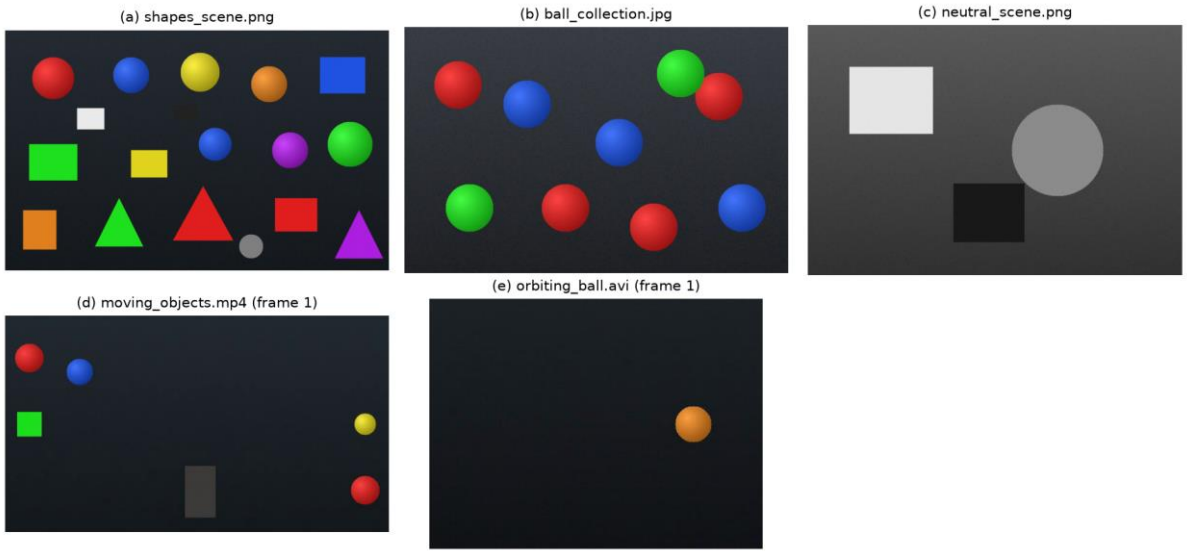


Figure 3: Sample Dataset Used for Testing

The images and the first frame of each video are shown; the users can also upload their own media.

9.2 Colour space conversion (BGR to HSV)

OpenCV loads images in BGR order. Each image or frame is converted to HSV with cv2.cvtColor(image, cv2.COLOR_BGR2HSV). In OpenCV, the hue channel ranges from 0 to 179 (the 0°–360° colour wheel divided by two), while saturation and value range from 0 to 255. Because hue describes the colour independently of brightness, a single hue range can describe a colour under different lighting, and minimum saturation and value thresholds can reject grey, white and very dark pixels.

9.3 HSV thresholding and colour presets

A binary mask is produced with cv2.inRange(hsv, lower, upper), which sets a pixel to 255 when all three channels lie within the range and to 0 otherwise. The following presets are provided; each can be fine-tuned from the interface:

Colour	Hue range	Saturation	Value
Red	0–10 and 170–179	120–255	70–255
Orange	11–20	120–255	100–255
Yellow	21–34	100–255	100–255
Green	35–85	60–255	50–255
Blue	95–128	80–255	50–255
Purple	129–160	60–255	50–255

Dual-range red detection: Red hues lie near 0 and also near 179, at the two ends of the hue circle. A single interval cannot cover both ends, so two masks are created and combined with a bitwise OR: mask = inRange(hsv, (0, S, V), (10, 255, 255)) OR inRange(hsv, (170, S, V), (179, 255, 255)). An automated test confirms that a single red range misses objects at one end of the hue circle, while the dual range detects both.

9.4 Morphological operations

The raw mask usually contains isolated noise pixels and small holes. Morphological operations with a 5 × 5 elliptical structuring element (adjustable from 3 to 11) are used to clean it:

- **Erosion:** shrinks white regions and removes small noise pixels.
- **Dilation:** grows white regions and fills small gaps.
- **Opening (erosion followed by dilation):** removes small noise while keeping the size of larger objects.
- **Closing (dilation followed by erosion):** fills small holes inside objects.

The final mask is obtained by applying opening followed by closing. The erosion and dilation results are also shown in the interface for comparison.

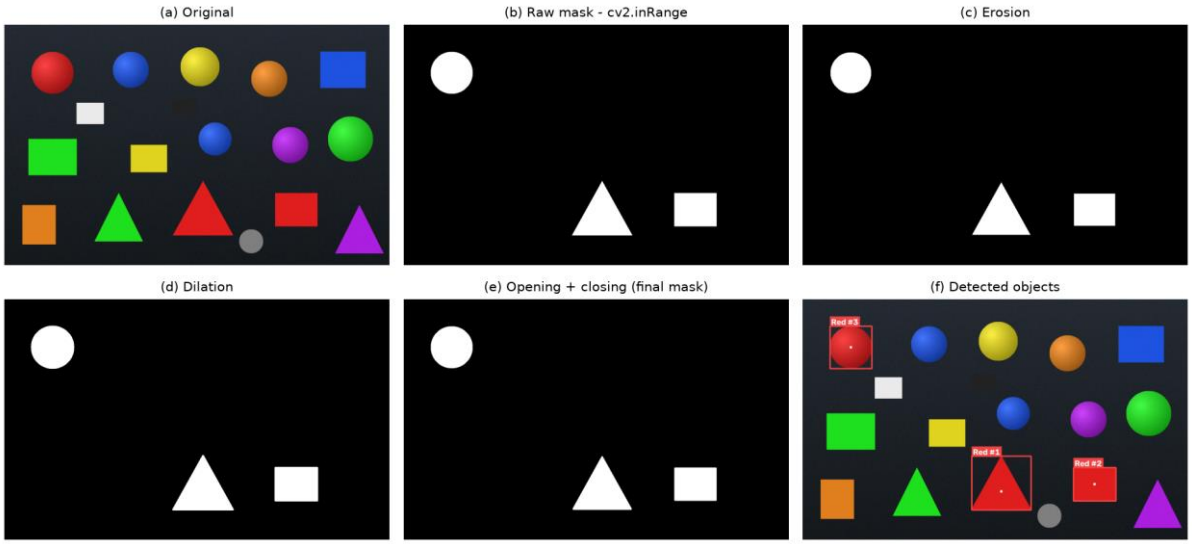


Figure 4: Stages of the Image Processing Pipeline (Red Preset)

The stages are generated from the real pipeline on shapes_scene.png: original image, raw mask, erosion, dilation, final mask and detected objects.

9.5 Contour detection and object localisation

The external contours of the cleaned mask are found with cv2.findContours using RETR_EXTERNAL and CHAIN_APPROX_SIMPLE. Contours with an area below the minimum area (300 px for images and 250 px for videos by default) are discarded as noise. For each remaining object the system calculates:

- **Bounding box:** x, y, width and height from cv2.boundingRect.
- **Area:** the contour area in pixels from cv2.contourArea.
- **Centroid:** from the image moments, cx = M10 / M00 and cy = M01 / M00 (the box centre is used if M00 is zero).

Bounding boxes, labels with the object number and area, and centroid markers are drawn on the output image. The segmented image is produced with a bitwise AND of the original image and the mask.

9.6 Centroid tracking algorithm

For videos, a simple centroid-based multi-object tracker is used. It is not a deep learning tracker; it links detections between consecutive frames purely by the distance between their centroids. For each frame:

1. Receive the list of centroids detected in the current frame.
2. Compute the Euclidean distance between every existing track and every new centroid.
3. Match pairs greedily, closest first, as long as the distance is below the maximum match distance (80 px by default).
4. Start a new track with a new ID for every unmatched detection.
5. Mark every unmatched track as disappeared; remove it when it has been missing for more than the allowed number of frames (15 by default).

Step 5 lets an object keep its ID through short missed detections such as brief occlusions. The tracker also stores the path of each object, which is used to draw motion trails and trajectory charts and to calculate the path length.

9.7 Video processing

OpenCV's VideoCapture reads the uploaded video frame by frame. Frames wider than 960 px are resized to keep processing fast. Each frame passes through the detection pipeline and the tracker, and the annotated frame, with boxes, IDs, trails and a status bar, is written to two output files: WebM (VP8), which plays directly in the browser, and MP4 (mp4v) for desktop players. Per-frame results (object count and processing time) are collected for the analytics and for the CSV export.

9.8 Result analysis

The system collects the detection results and presents the following measured information:

- Number of detected objects and the area of each object.
- Colour coverage: the percentage of image pixels in the final mask.
- Colour distribution of the whole image, grouped into colour families from the HSV values.
- Hue spectrum (histogram of hue values of coloured pixels).
- For video: frames processed, frames with detections, detection frequency, unique track IDs, objects per frame, processing time per frame, trajectories and track lifetimes.

All values are calculated from the actual run. When no run has been made, the Analytics page shows an empty state instead of placeholder values.

10. Implementation Details

10.1 Development environment

Python virtual environment: The project runs in a Python 3.12 virtual environment on Windows. The dependencies are listed in requirements.txt and installed with pip install -r requirements.txt.

Streamlit application: The application is started with streamlit run app.py and opens in the browser at http://localhost:8501. Uploaded files and processed outputs are stored in a temporary folder for each session, and folders older than 12 hours are cleaned up automatically.

10.2 Main modules

The application is organised into the following Python modules:

- **Colour Module (color_ranges.py):** Defines the HSV presets, builds the dual red range and custom ranges, and converts between colour formats.
- **Image Processing Module (image_processing.py):** Loads and validates images, converts to HSV, creates the mask, applies morphology, finds contours and draws the detections.

- **Tracking Module (object_tracking.py):** Implements the centroid tracker and produces the track summary.
- **Video Processing Module (video_processing.py):** Reads video information, creates browser previews, processes every frame and writes the output videos.
- **Analytics Module (analytics.py):** Computes the colour distribution and hue histogram and builds the Plotly and Matplotlib charts.
- **Components (components/):** Session state, sidebar navigation, colour picker and reusable interface elements.
- **Views (views/):** The five pages: Overview, Image Analysis, Video Tracking, Analytics & Insights and About Project.
- **User Interface Entry Point (app.py):** Page configuration, theme loading, navigation and routing.

10.3 Application features

The Streamlit application provides the following features:

- Overview page with project introduction and live session statistics.
- Separate Image Analysis and Video Tracking pages.
- Image upload (JPG, JPEG, PNG) and video upload (MP4, AVI, MOV), with sample demo mode.
- Six colour presets with fine-tune sliders and a custom HSV range.
- Adjustable kernel size, iterations, minimum area, match distance and lost-ID frames.
- Side-by-side original, mask and processed image, with mask and morphology tabs.
- Bounding boxes, labels, centroids, track IDs and motion trails.
- Object table with area, bounding box and centroid of each object.
- Analytics & Insights page with charts calculated from the latest runs and a session history.
- Download options for the processed image, mask, segmented image, tracked video (WebM and MP4) and CSV reports.
- Clear error messages for invalid or empty files and an information message when no object is found.
- About page describing the techniques, the architecture and the limitations.

The interface uses a consistent dark theme and is designed to make the processing steps simple to follow.

11. Testing and Observation

11.1 Application test cases

The following test cases were run against the application in a real browser using the sample images and videos. All 22 end-to-end checks passed; the main cases are listed below.

Test case	Expected behaviour	Observation
Run the sample image demo (shapes_scene.png, Red)	Three red objects should be detected.	3 objects detected; coverage 4.81%; boxes and centroids drawn.
Select Blue on shapes_scene.png	Three blue objects should be detected.	3 objects detected.
Select Green on ball_collection.jpg	Two green balls should be detected.	2 objects detected.
Upload ball_collection.jpg and select Red	Four red balls should be detected.	4 objects detected.
Analyse neutral_scene.png	No object should be found and a message shown.	Information message shown; no false detections.
Upload a non-image file renamed as .png	A clear error message should be shown.	Error message shown; the application kept running.
Run the sample video demo (moving_objects.mp4, Red)	Two red balls should keep two IDs.	200 frames processed; 2 unique IDs maintained.
Upload orbiting_ball.avi	Video information should be read.	120 frames at 30 fps shown.
Download the processed image and videos	PNG, WebM and MP4 files should download.	All three files downloaded.
Open Analytics & Insights after the runs	Image and video metrics should be shown.	Metrics and charts displayed from the latest runs.
Open the application at mobile width	The layout should not overflow sideways.	No horizontal overflow.

11.2 Automated unit tests

The core modules are tested with pytest. All 34 tests passed (34 passed in 5.93 s).

Test file	Tests	What is tested	Result
test_image_processing.py	17	Red at both hue ends, blue and green detection, multiple objects, no-match case, invalid input, opening and closing, minimum area, resizing, colour distribution.	17 passed
test_tracking.py	6	Consistent IDs, survival through short occlusion, removal after the limit, new IDs for far or new objects, path length.	6 passed
test_video_processing.py	11	Video information, AVI and MOV input, tracking of two red balls, single orange ball, readable outputs, no detections, resizing, invalid and empty files, preview.	11 passed
Total	34		34 passed

11.3 Image detection results

The detected object counts were compared with the ground truth for each sample image and colour:

Image	Colour	Expected	Detected	Match	Coverage	Time
shapes_scene.png	Red	3	3	Yes	4.81%	29.4 ms
shapes_scene.png	Orange	2	2	Yes	2.52%	8.5 ms
shapes_scene.png	Yellow	2	2	Yes	2.37%	7.9 ms
shapes_scene.png	Green	3	3	Yes	4.90%	8.7 ms
shapes_scene.png	Blue	3	3	Yes	3.83%	7.3 ms
shapes_scene.png	Purple	2	2	Yes	2.37%	8.5 ms
ball_collection.jpg	Red	4	4	Yes	7.63%	5.1 ms
ball_collection.jpg	Blue	3	3	Yes	5.93%	5.0 ms
ball_collection.jpg	Green	2	2	Yes	3.91%	4.4 ms
neutral_scene.png	All six presets	0	0	Yes	0.00%	–

All 15 image–colour combinations matched the ground truth. The first run (29.4 ms) includes start-up overhead; the remaining runs took between 4.4 and 8.7 ms.

11.4 Video tracking results

Video	Colour	Frames	Frames with detections	Expected objects	Unique IDs	Avg time / frame
moving_objects.mp4	Red	200	200 (100.0%)	2	2	9.35 ms
moving_objects.mp4	Blue	200	200 (100.0%)	1	1	8.82 ms
moving_objects.mp4	Purple	200	0 (0.0%)	0	0	8.10 ms
orbiting_ball.avi	Orange	120	120 (100.0%)	1	1	10.67 ms

The number of unique IDs matched the expected number of objects in every case. For Purple, which is not present in the video, no objects were detected. The track details of the two red balls are:

Track ID	First frame	Last frame	Frames seen	Path length (px)
1	1	200	191 of 200	585.2
2	1	200	200 of 200	578.0

Track 1 was not detected in 9 frames while it was hidden, but it kept the same ID when it reappeared because the gap was shorter than the 15-frame limit.

11.5 Output screenshots

The screenshots below were captured from the running application.

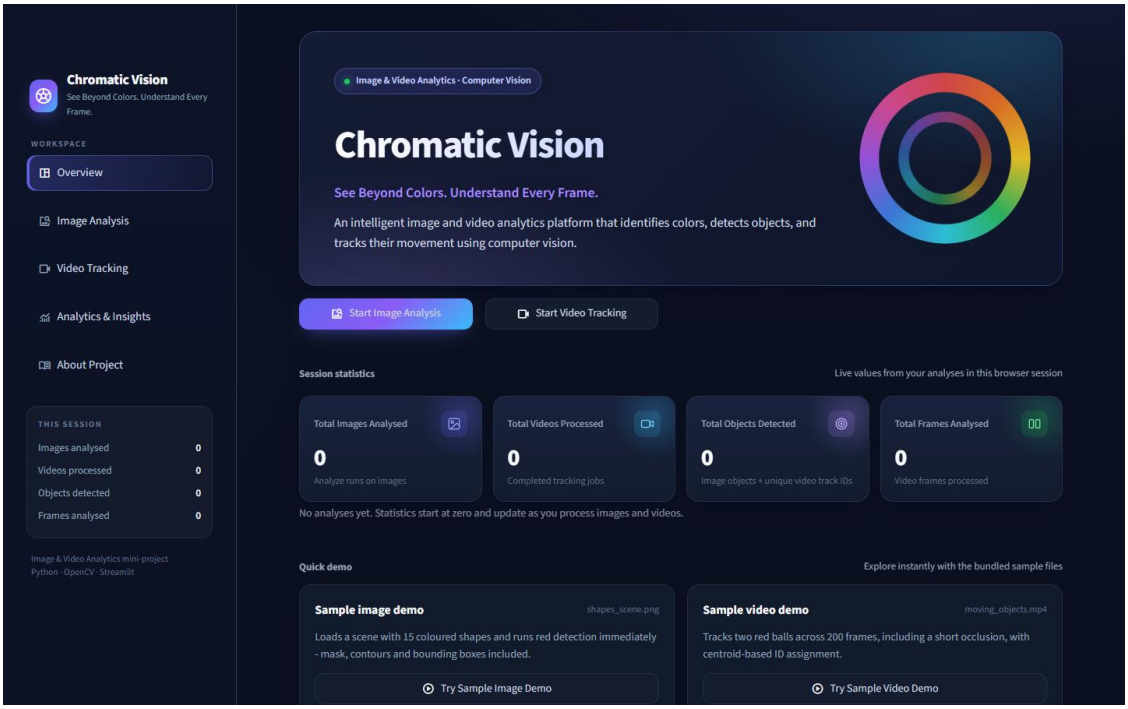


Figure 5: Overview Page

The overview page introduces the project and shows live session statistics.

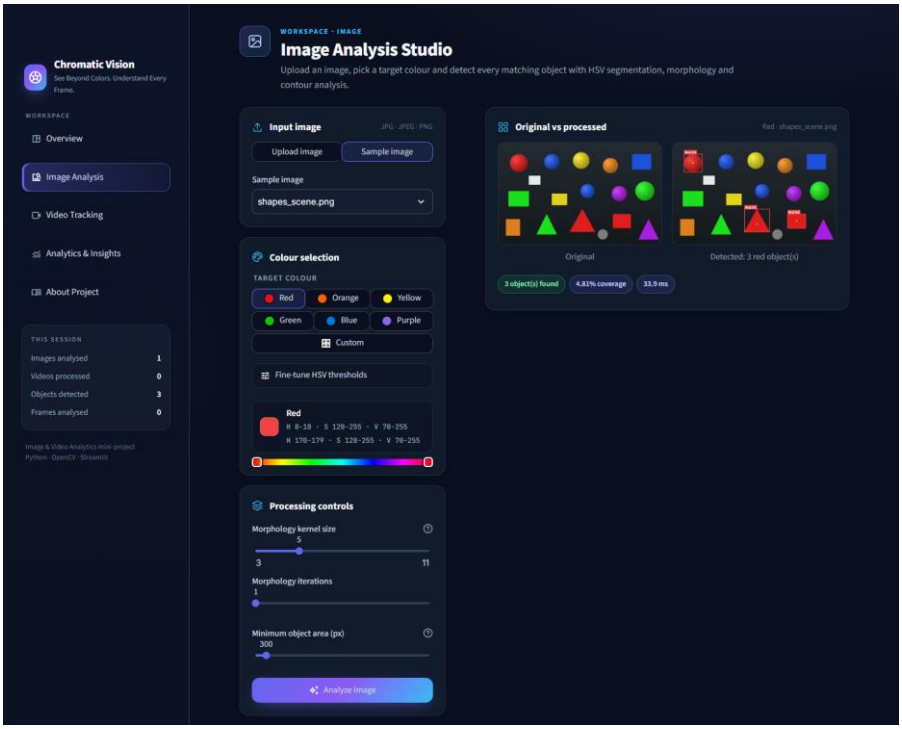


Figure 6: Image Analysis Page with Colour Selection

The user selects an image, a colour preset and the processing parameters; the sample image and its processed version are shown side by side.

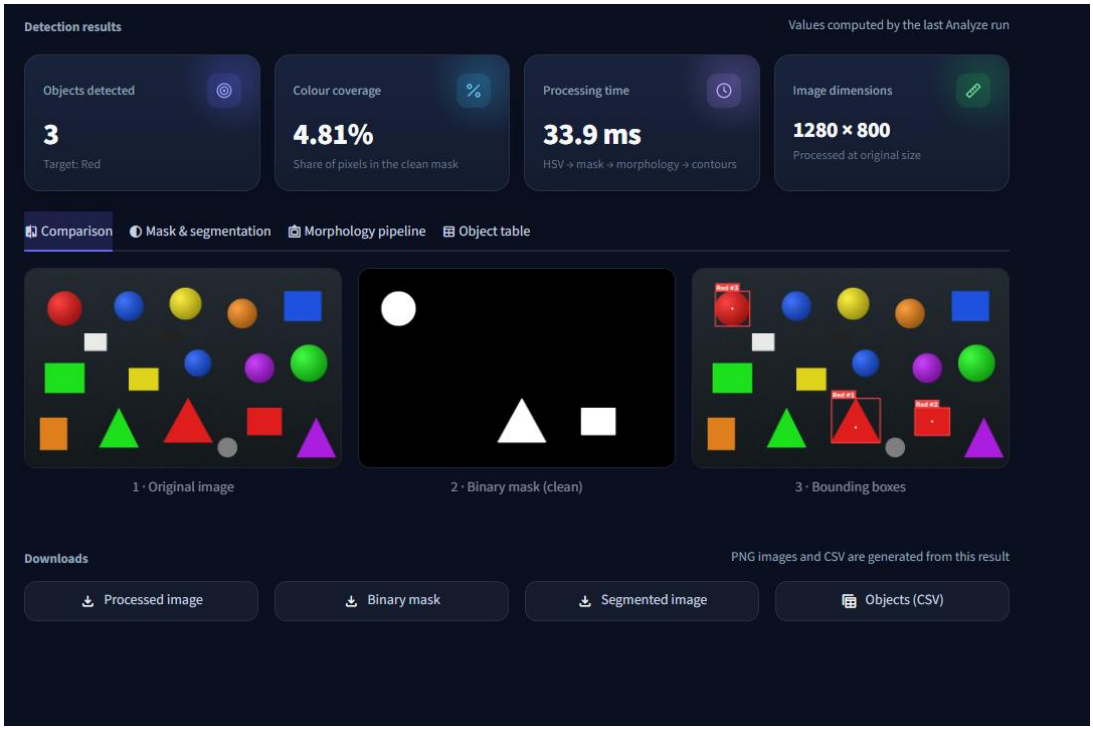


Figure 7: Image Detection Results

The system detected 3 red objects with 4.81% coverage and showed the original image, binary mask and bounding boxes with download options.

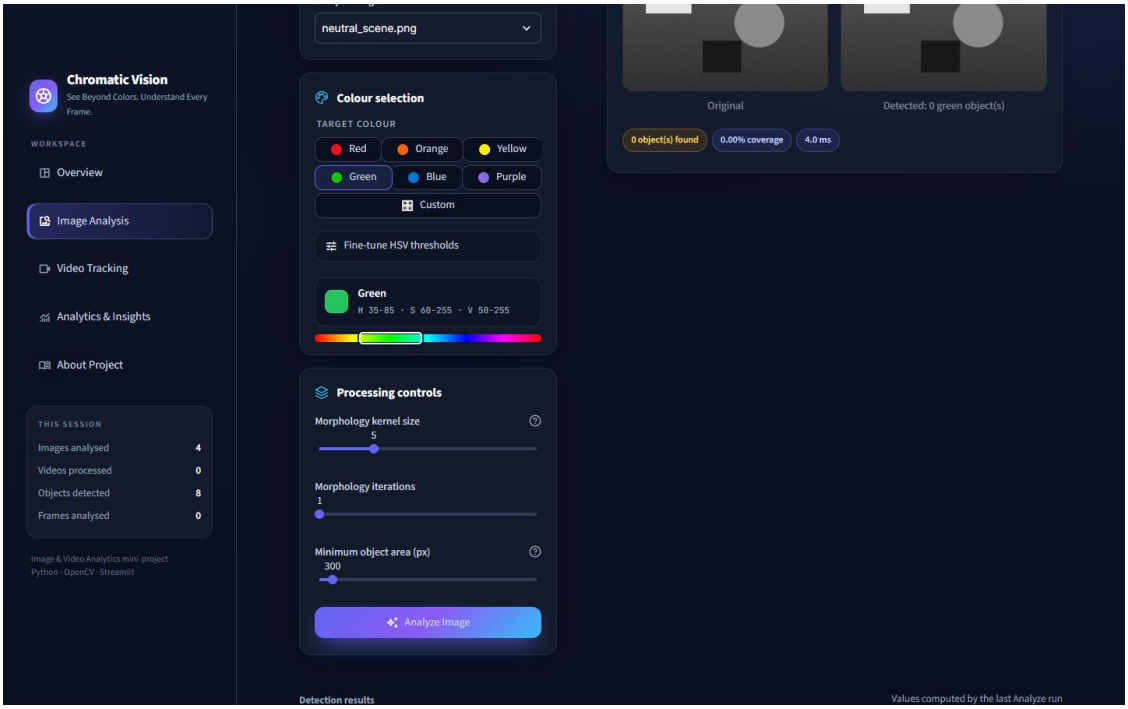


Figure 8: No Matching Colour Detected

For neutral_scene.png no green object was found, and the system displayed an information message instead of false detections.

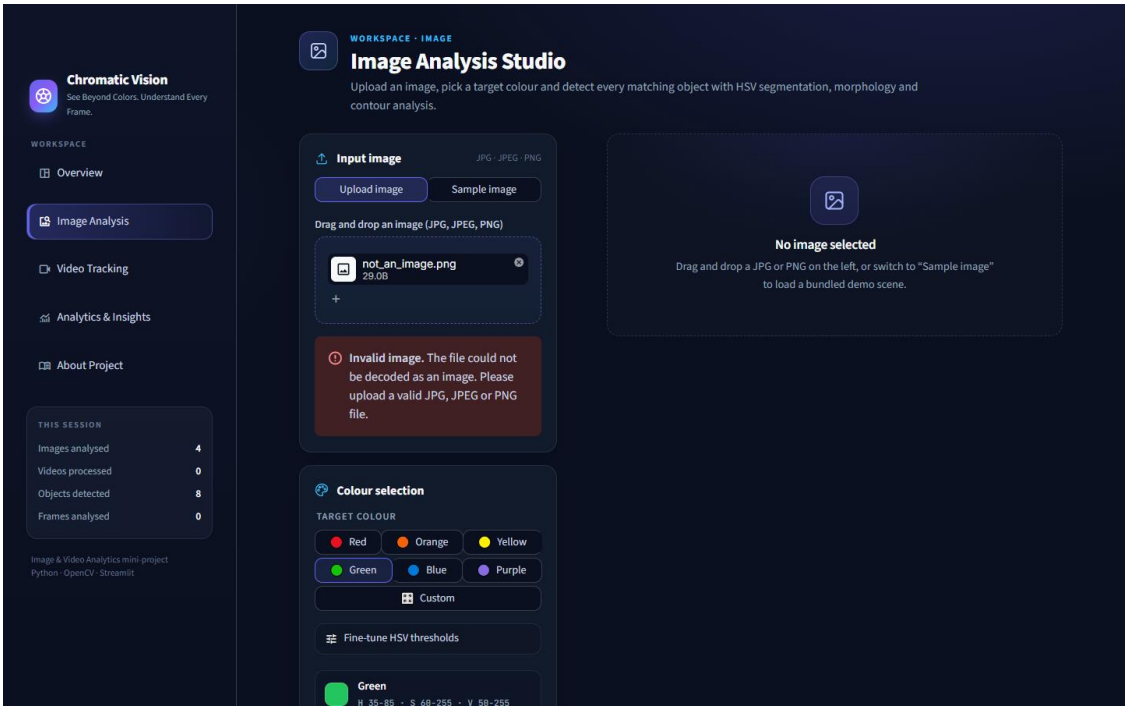


Figure 9: Invalid File Handling

An invalid file was rejected with a clear error message.

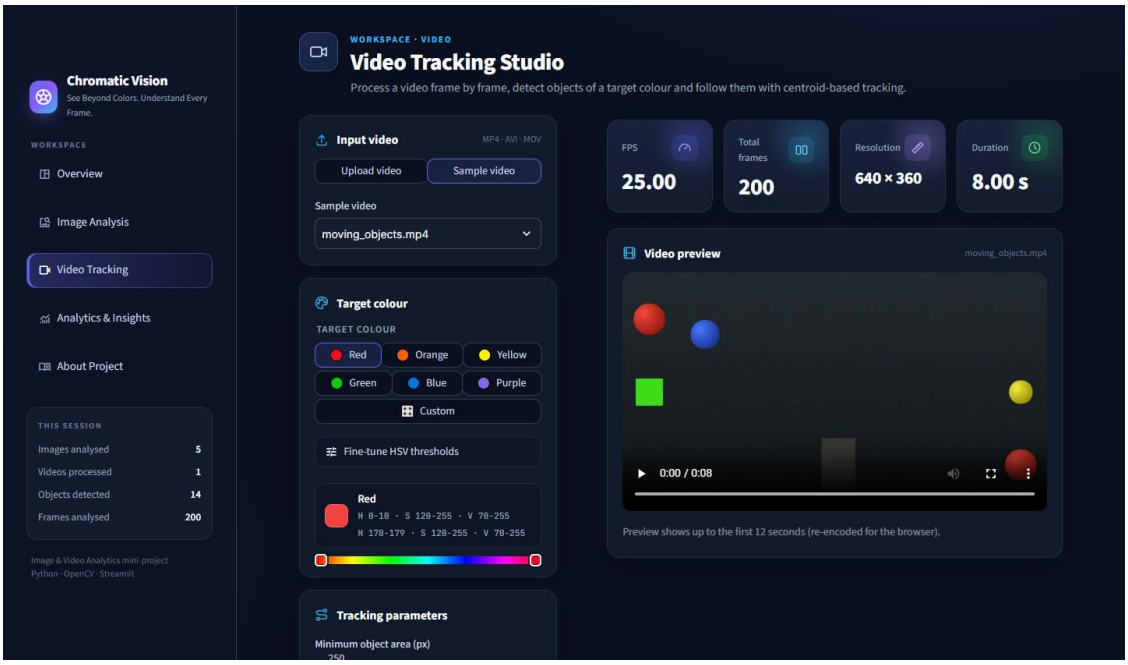


Figure 10: Video Tracking Page

The video information (25 fps, 200 frames, 640 × 360, 8.00 s) and a preview are shown before processing.

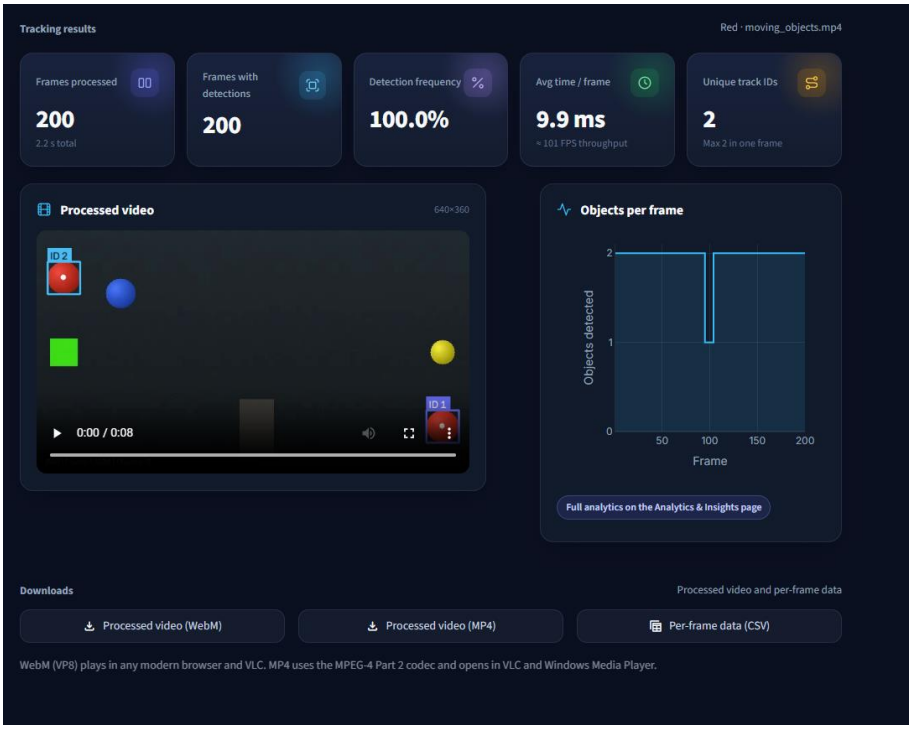


Figure 11: Video Tracking Results

The system processed 200 frames at 9.9 ms per frame and tracked the two red balls with 2 unique IDs.

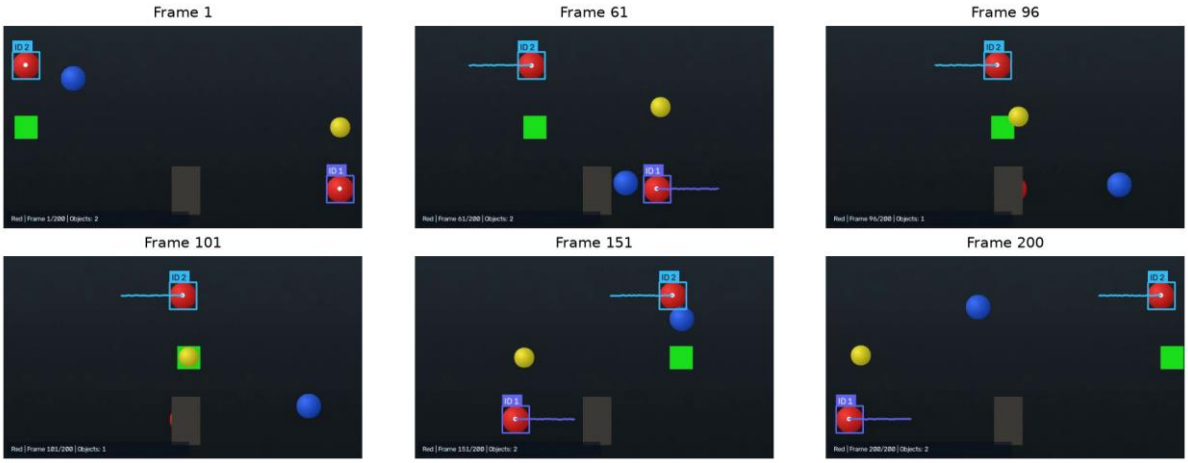


Figure 12: Tracked Frames from the Output Video

Each red ball keeps its ID and motion trail; between frames 96 and 101 track 1 is briefly hidden and keeps the same ID when it reappears.

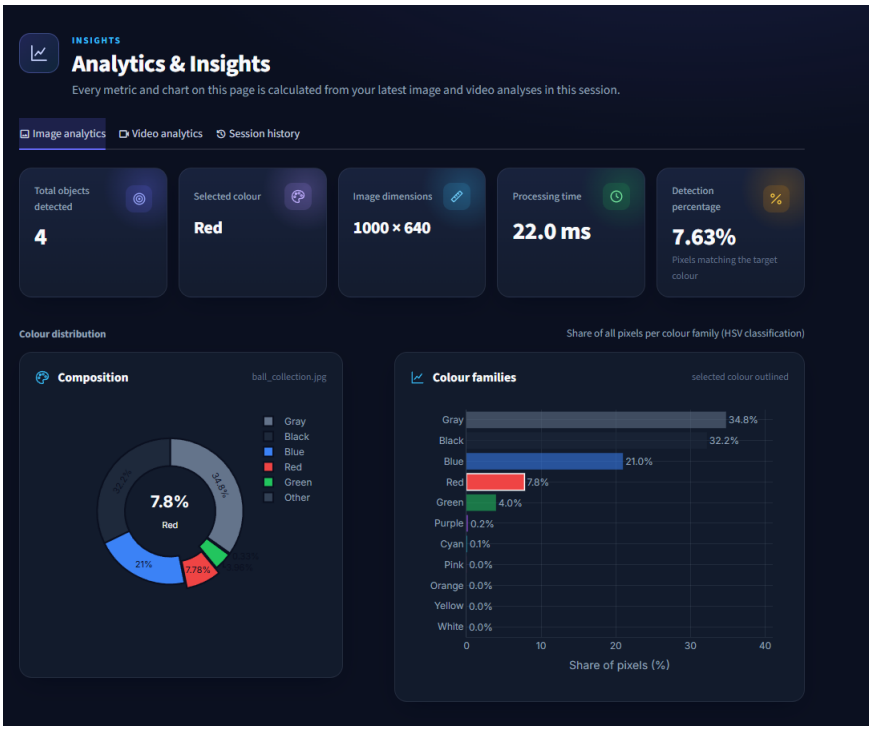


Figure 13: Analytics – Image Colour Distribution

The colour composition and colour families of the analysed image are calculated from its pixels.

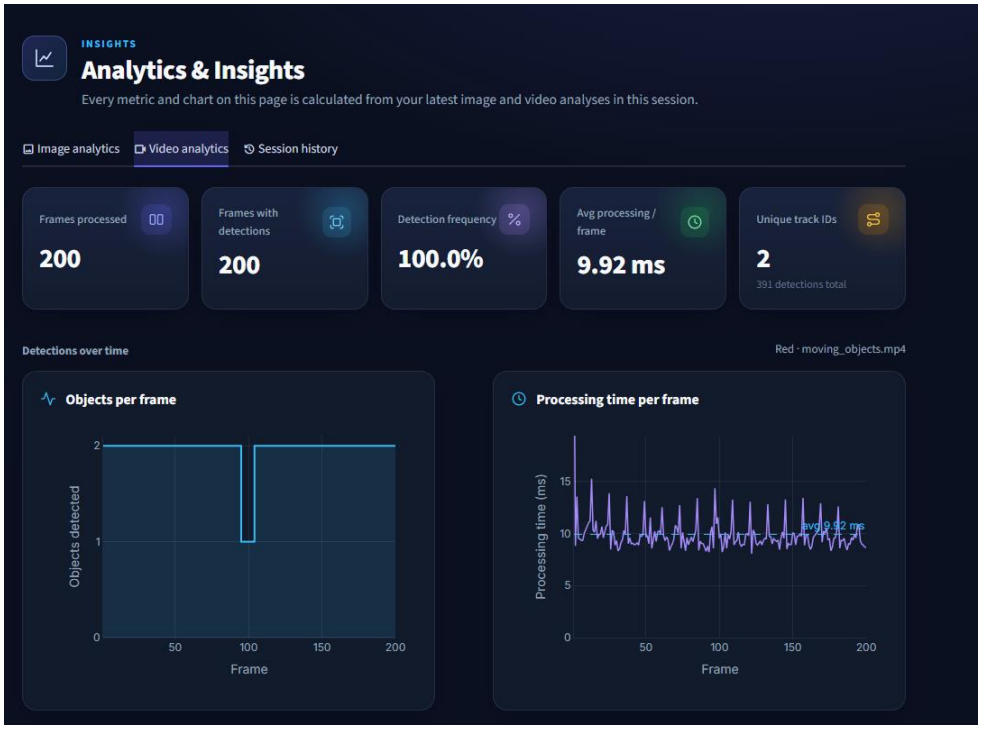


Figure 14: Analytics – Video Detections and Processing Time

The number of objects and the processing time are plotted for every frame of the video.

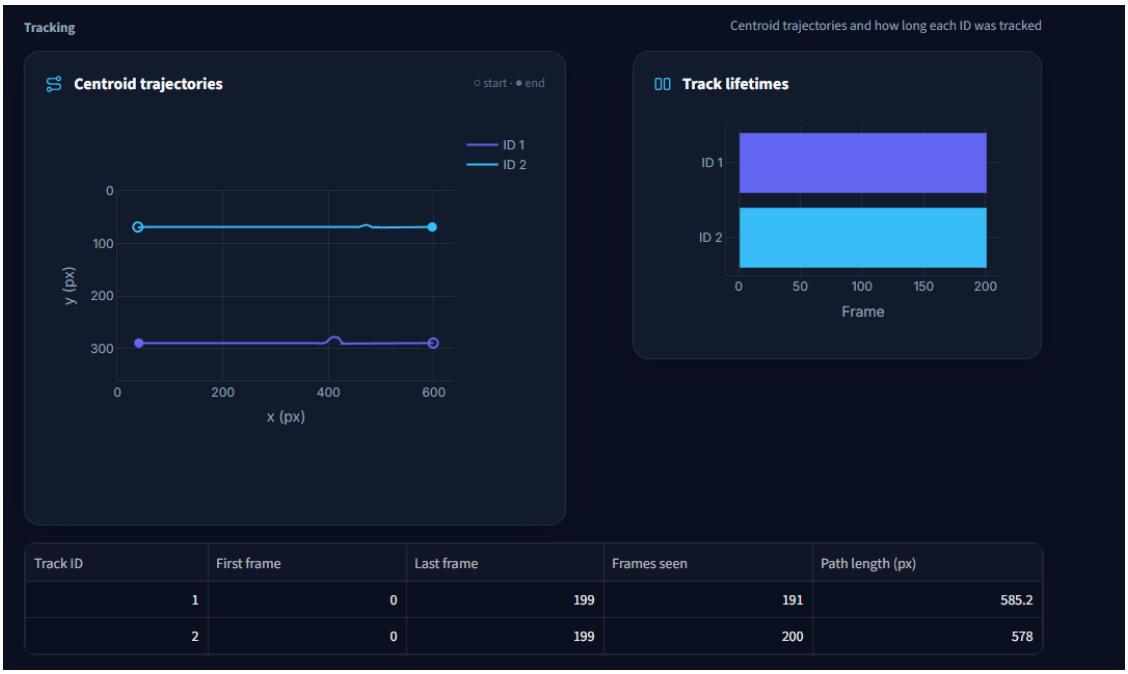


Figure 15: Analytics – Trajectories and Track Lifetimes

The centroid path and the lifetime of each track are shown with the track summary table.

12. Limitations

- Detection depends only on colour, so different objects of the same colour cannot be told apart.
- Strong lighting changes, shadows, reflections and colour casts may move pixels outside the HSV range.
- Low saturation colours (pastels) and colours close to white, grey or black are difficult to separate.
- Objects of the same colour that touch or overlap are merged into a single contour.
- Background regions of the target colour are detected as objects.

- The centroid tracker can swap IDs when two objects cross closely, and objects hidden for longer than the allowed frames receive a new ID.
- The presets are tuned on the bundled synthetic samples; real-world scenes may need fine-tuning with the sliders.
- Long or high-resolution videos take more time to process, and the system processes uploaded files rather than live camera streams.

13. Conclusion and Future Enhancements

13.1 Conclusion

Chromatic Vision demonstrates the application of classical image processing techniques to colour detection and object tracking. The system converts images and video frames to the HSV colour space, segments the target colour with HSV thresholding (including dual-range red), cleans the mask with morphological operations, and locates objects with contour analysis, bounding boxes and centroids. A centroid tracker keeps consistent IDs for moving objects in videos.

On the sample dataset the detected counts matched the ground truth for all 15 image–colour combinations, every sample video produced the expected number of track IDs, and the 200-frame video was processed at about 9.3 ms per frame on a CPU. The Streamlit application presents the results with real analytics and downloadable outputs, and the system was verified with 34 unit tests and 22 end-to-end checks.

This project provides practical experience in colour spaces, image segmentation, morphological processing, contour analysis, object tracking, video analytics, testing and web application development.

13.2 Future enhancements

The project can be improved in the following ways:

- **Live Camera Input:** Process webcam or CCTV streams in real time.
- **Multiple Colours at Once:** Detect and track several colours in a single pass.
- **Adaptive Thresholds:** Adjust the HSV ranges automatically to the lighting of each scene.
- **Better Tracking:** Use the Hungarian algorithm or a Kalman filter to reduce ID swaps when objects cross.
- **Shape Classification:** Classify objects as circles, squares or triangles using contour approximation.
- **Object Separation:** Use the watershed algorithm to separate touching objects of the same colour.
- **Real-World Evaluation:** Tune and evaluate the presets on a labelled set of real photographs and videos.
- **Cloud Deployment:** Deploy the application online so that it can be used without installation.

References

1. Gonzalez, R. C. and Woods, R. E. Digital Image Processing, 4th ed. Pearson, 2018.
2. Smith, A. R. “Color Gamut Transform Pairs.” ACM SIGGRAPH Computer Graphics, 12(3), 1978, pp. 12–19.
3. Suzuki, S. and Abe, K. “Topological Structural Analysis of Digitized Binary Images by Border Following.” Computer Vision, Graphics, and Image Processing, 30(1), 1985, pp. 32–46.
4. OpenCV Documentation – Changing Colorspaces. https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html
5. OpenCV Documentation – Morphological Transformations. https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html
6. OpenCV Documentation – Contours: Getting Started. https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html

7. OpenCV Documentation – Contour Features (moments, bounding rectangle). https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html
8. OpenCV Documentation – VideoCapture Class. https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html
9. Streamlit Documentation. https://docs.streamlit.io/
10. Plotly Python Documentation. https://plotly.com/python/
11. Matplotlib Documentation. https://matplotlib.org/stable/
12. NumPy Documentation. https://numpy.org/doc/
13. Pillow Documentation. https://pillow.readthedocs.io/
14. pytest Documentation. https://docs.pytest.org/
15. Python Documentation. https://docs.python.org/3/